ZK From Scratch

build a zk-STARK with minimal external libraries

314552031 陳宏彰

2026-06-09

1. Zero Knowledge

1.1 Types of ZK

	zk-SNARK	zk-STARK
cryptographic support	elliptic curve, pairings	hash functions
prove size	small, $O(1)$	~100KiB, $O(\log(n))$
verifier costs (gas on chain)	low	high
quantum resistance	no	yes (by changing the hash function)

2. Finite Field

2.1 Finite Field

All operations are performed in a finite field $\mathbb{F}_M$

$$\mathbb{F}_5 = \{0, 1, 2, 3, 4\}$$

$$1 + 2 = 3 (\mathrm{mod}\, 5)$$

$$2 \times 4 = \not{8} = 3 (\mathrm{mod}\, 5)$$

2.2 Generator

A generator is an element of $\mathbb{F}_5^*$ and its powers generate all the elements in $\mathbb{F}_5^*$

```
[2**i % 5 for i in range(5-1)]
```

[1, 2, 4, 3]

2.2 Generator

A generator is an element of $\mathbb{F}_5^*$ and its powers generate all the elements in $\mathbb{F}_5^*$

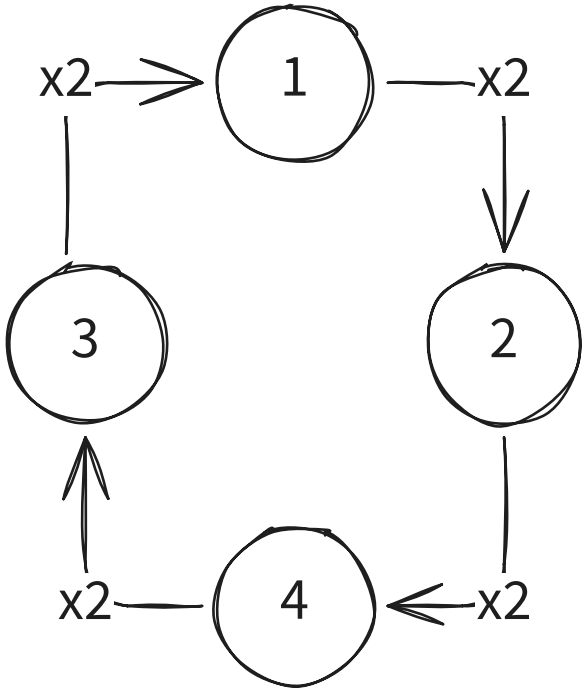


Figure 1: $\mathbb{F}_5^*$ with generator 2

3. The VM

- 2 registers `reg1` and `reg2`
- 4+1 operations: `set`, `swap`, `add`, `mul` and `nop`

- 2 registers `reg1` and `reg2`
- 4+1 operations: `set`, `swap`, `add`, `mul` and `nop`

For example $(1 + 2) \times 3 + 6$ is compiled into

```
set 1 // reg2 = 1
swap  // reg1, reg2 = reg2, reg1
set 2 // reg2 = 2
add   // reg1 = reg1 + reg2
set 3 // reg2 = 3
mul   // reg1 = reg1 * reg2
set 6 // reg2 = 6
add   // reg1 = reg1 + reg2
```

after executing the program, the trace is

reg1	reg2	input	set	swap	add	mul	nop
0	0	1	1	0	0	0	0
0	1	0	0	1	0	0	0
1	0	2	1	0	0	0	0
1	2	0	0	0	1	0	0
3	2	4	1	0	0	0	0
3	4	0	0	0	0	1	0
12	4	6	1	0	0	0	0
12	6	0	0	0	1	0	0
18	6	0	0	0	0	0	1

4. What is in the Proof?

4.1 The Merkle Roots of the Trace Polynomials

1. Use Lagrange interpolation to find the polynomials that pass through the points in domain $\mathbb{G}$, where $\mathbb{G} \subset \mathbb{F}_M$, $f_{\text{reg1}}(x)$, $f_{\text{reg2}}(x)$, $f_{\text{input}}(x)$ etc.

$$f_{\text{reg1}}(g^i) = \text{reg1}[i]$$

4.1 The Merkle Roots of the Trace Polynomials

1. Use Lagrange interpolation to find the polynomials that pass through the points in domain $\mathbb{G}$, where $\mathbb{G} \subset \mathbb{F}_M$, $f_{\text{reg1}}(x)$, $f_{\text{reg2}}(x)$, $f_{\text{input}}(x)$ etc.

$$f_{\text{reg1}}(g^i) = \text{reg1}[i]$$

2. Evaluate those $f_k(x)$ in a larger domain $\mathbb{L}$

4.1 The Merkle Roots of the Trace Polynomials

1. Use Lagrange interpolation to find the polynomials that pass through the points in domain $\mathbb{G}$, where $\mathbb{G} \subset \mathbb{F}_M$, $f_{\text{reg1}}(x)$, $f_{\text{reg2}}(x)$, $f_{\text{input}}(x)$ etc.

$$f_{\text{reg1}}(g^i) = \text{reg1}[i]$$

2. Evaluate those $f_k(x)$ in a larger domain $\mathbb{L}$
3. Generate Merkle trees for those values and commit the root to the verifier.

4.2 The Trace Size

The trace size N can be used to construct the vanish polynomials

$$u_k(x) = \prod_{g \in \mathbb{G}_k} (x - g)$$

where G_k is the domain of $c_k(x)$, which are public parameters.

5. How to Verify

5.1 Challenge Initialization

The verify randomly chooses some number β_k.

5.1 Challenge Initialization

The verify randomly chooses some number β_k. Prover then constructs the composition polynomial

$$p(x) = \sum_k \beta_k \frac{c_k(x)}{u_k(x)}$$

5.1 Challenge Initialization

The verify randomly chooses some number β_k. Prover then constructs the composition polynomial

$$p(x) = \sum_k \beta_k \frac{c_k(x)}{u_k(x)}$$

Then commit the Merkle root of $p(x)$ in $\mathbb{L}$ to the verifier.

5.2 Check the composition polynomial

The verifier then generates a sequence of $z \in \mathbb{L}$, and the prover responses $p(z)$ and $c_k(z)$. The verifier can check if

$$p(z) = \sum_k \beta_k \frac{c_k(z)}{u_k(z)}$$

$$c_{\text{r1\_add}}(x) = f_{\text{add}}(x) \cdot (f_{\text{r1}}(x \cdot g) - (f_{\text{r1}}(x) + f_{\text{r2}}(x))) = 0 \forall x \in \mathbb{G}_k$$

$$u_k(x) = \Pi_{g \in \mathbb{G}_k}(x - g) = 0 \forall x \in \mathbb{G}_k$$

Note: The verifier can calculate $u_k(z)$ by itself

5.3 Check the degree of $p(x)$

1. The verifier randomly chooses some α_j and some $z \in \mathbb{L}$

[1] Fast Reed-Solomon Interactive Oracle Proof of Proximity

5.3 Check the degree of $p(x)$

1. The verifier randomly chooses some α_j and some $z \in \mathbb{L}$
2. The prover uses FRI protocol[1] to fold $p(x)$ into a constant

[1] Fast Reed-Solomon Interactive Oracle Proof of Proximity

5.3 Check the degree of $p(x)$

1. The verifier randomly chooses some α_j and some $z \in \mathbb{L}$
2. The prover uses FRI protocol[1] to fold $p(x)$ into a constant
3. The verifier checks if the folding is correct and the final constant is identical among all input z

[1] Fast Reed-Solomon Interactive Oracle Proof of Proximity

6. Thanks for Listening!

7. Non-Interactive

replace all randomly sampled values with hash

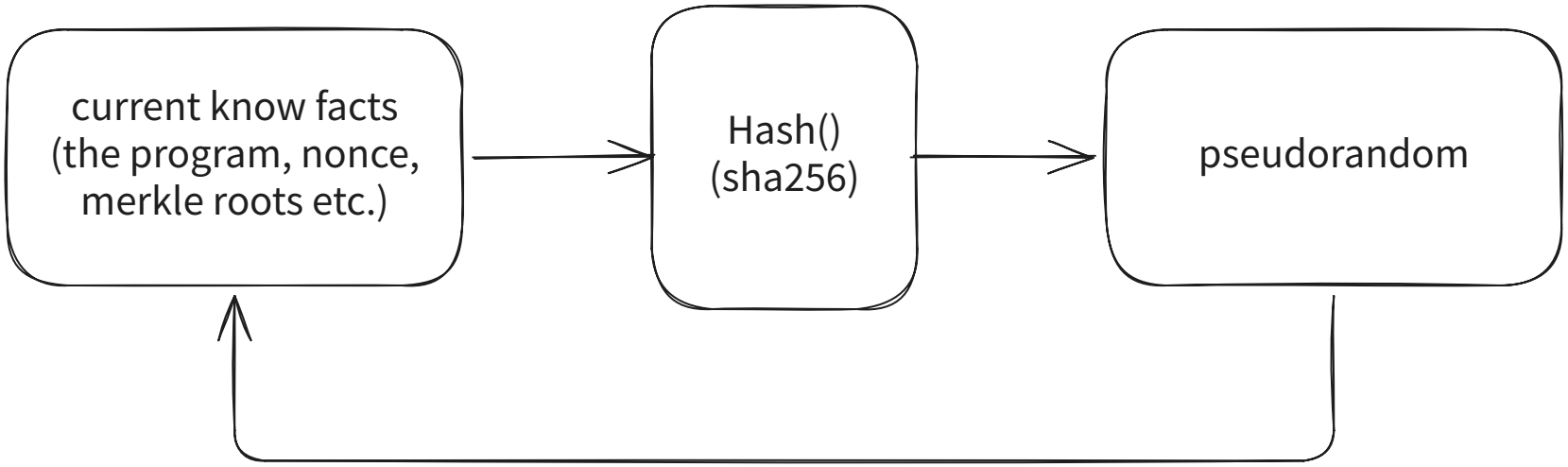


Figure 2: Fiat–Shamir heuristic

then the prover can generate z, α_j and β_k by itself and generate the proof.

8. Constraints

$\mathbb{G}_k$ is $\mathbb{G}$

- $\text{set} \cdot (\text{set} - 1)$
- $\text{swap} \cdot (\text{swap} - 1)$
- $\text{add} \cdot (\text{add} - 1)$
- $\text{mul} \cdot (\text{mul} - 1)$
- $\text{nop} \cdot (\text{nop} - 1)$
- $\text{add} + \text{mul} + \text{nop} + \text{set} + \text{swap} - 1$
- $\text{input} \cdot (1 - \text{set})$

$\mathbb{G}_k$ is $\mathbb{G}$ except the last element

- $\text{set} \cdot (\text{r1\_next} - \text{r1\_curr})$
- $\text{set} \cdot (\text{r2\_next} - \text{input})$

- $\text{swap} \cdot (\text{r1\_next} - \text{r2\_curr})$
- $\text{swap} \cdot (\text{r2\_next} - \text{r1\_curr})$
- $\text{add} \cdot (\text{r1\_next} - (\text{r1\_curr} + \text{r2\_curr}))$
- $\text{add} \cdot (\text{r2\_next} - \text{r2\_curr})$
- $\text{mul} \cdot (\text{r1\_next} - \text{r1\_curr} \cdot \text{r2\_curr})$
- $\text{mul} \cdot (\text{r2\_next} - \text{r2\_curr})$
- $\text{nop} \cdot (\text{r1\_next} - \text{r1\_curr})$
- $\text{nop} \cdot (\text{r2\_next} - \text{r2\_curr})$

$\mathbb{G}_k$ is the first element in $\mathbb{G}$

- $\text{r1\_curr} - \text{r1}_{\text{first}}$
- $\text{r2\_curr} - \text{r2}_{\text{first}}$

9. Finite Field

A finite field $\mathbb{F}$ is a set of elements with two operations (addition and multiplication). After performing the operations, the result is still in the set.

$$\mathbb{F} = \{0, 1, 2, 3, 4\}$$

$$1 + 2 = 3$$

$$2 \times 4 = \not{8} = 3$$

A finite field $\mathbb{F}$ is a set of elements with two operations (addition and multiplication). After performing the operations, the result is still in the set.

$$\mathbb{F} = \{0, 1, 2, 3, 4\}$$

How about minus and division?

$$1 - 2 = ?$$
$$1 \div 2 = ?$$

A finite field $\mathbb{F}$ is a set of elements with two operations (addition and multiplication). After performing the operations, the result is still in the set.

$$\mathbb{F} = \{0, 1, 2, 3, 4\}$$

$$1 - 2 = x$$
$$1 = 2 + x$$
$$\overset{\text{mod } 5}{\Longrightarrow} 1 + 5 = 6 = 2 + x$$
$$\Rightarrow x = 4$$
$$\Rightarrow \mathbf{1 - 2 = 4}$$

A finite field $\mathbb{F}$ is a set of elements with two operations (addition and multiplication). After performing the operations, the result is still in the set.

$$\mathbb{F} = \{0, 1, 2, 3, 4\}$$

$$1 \div 2 = y$$

$$1 = 2 \times y$$

$$\overset{\text{mod } 5}{\Longrightarrow} 1 + 5 = 6 = 2y$$

$$\Rightarrow y = 3$$

$$\Rightarrow \mathbf{1 \div 2 = 3}$$

10. Fermat’ s Little Theorem

If p is not divisor of a, then

$$a^{p-1} = 1 (\mathrm{mod}\, p)$$

If p is a prime, the theorem holds for all a in $\mathbb{F}_p^*$. Then all elements can be generated by a single element g (generator) by performing multiplication.

```
[2**i % 5 for i in range(5-1)]
```

[1, 2, 4, 3]

10.1 Subgroups of $\mathbb{F}$

A subgroup $\mathbb{G}$ of $\mathbb{F}$ is a subset of $\mathbb{F}$ that is closed under the operations. For example, if we take the generator $g = 2$ in $\mathbb{F}_{17}$, we can generate a subgroup $\mathbb{G}$ with 8 elements:

```
set([2**i % 17 for i in range(1, 17)])
```

{1, 2, 4, 8, 9, 13, 15, 16}

10.1 Subgroups of $\mathbb{F}$

$$M = 2^{2^k} + 1$$

- Goldilocks Field: $M = 2^{64} - 2^{32} + 1$
- Mini Goldilocks Field: $M = 2^{31} - 2^{27} + 1$
- Fermat Primes: 3, 5, 17, 257, 65535

pros:

- size of $\mathbb{F}^*$ can be divided by many powers of 2
- can accelarate some operations like modulo

10.2 How to choose a proper $\mathbb{G}$?

Support we want to choose a subgroup with size $N = 4$ in $\mathbb{F}_{17}^*$.
First we need to know one of the generator of $\mathbb{F}_{17}^*$, which is $\omega = 3$. Then with the following fomula, we will get the generator of $\mathbb{G}$

$$g = \omega^{\frac{M-1}{N}} = 3^{\frac{16}{4}} = 13 (\mathrm{mod}\, 17)$$

```
set([13**i % 17 for i in range(4)])
```

{1, 4, 13, 16}

11. Factor Throrem

If a is the root of a polynomial $f(x)$, then $f(x)$ can be divided by $x - a$ without remainder.

$$f(x) = x^3 - 9x^2 + 26x - 24$$

$$f(2) = f(3) = f(4) = 0$$

$$\Rightarrow \frac{f(x)}{x-2} = x^2 - 7x - 5$$

$$\Rightarrow \frac{f(x)}{x-5} = x^2 - 4x + 6 \ldots \quad 6$$

not divisible

12. Interpolation theorem

For any $n+1$ data points $(x_i, y_i), i = 0, 1, ..., n$, there exists a unique polynomial $f(x)$ with degree at most n such that $f(x_i) = y_i$ for all i.

In other words, if a polynomial has degree higher than n, it has extra degrees of freedom to pass other points.

TODO: a graph here

13. Generate the Proof

13.1 The Finite Field

- $\mathbb{F}_{65537}^*$
- $\omega = 3$
- size is $63356 = 2^{16}$

13.2 Trace Polynomials

Then find polynomials that output each column of the trace when evaluated in $\mathbb{G}$ (a subgroup of $\mathbb{F}$). First we need to expand the length of trace to the size of $\mathbb{G}$ (16) by adding nop at the end.

$$g = 64, \mathbb{G} = [1, 64, 4096, 65533, 65281, 49153, 16, 1024,$$
$$65536, 65473, 61441, 4, 256, 16384, 65521, 64513]$$

$$f_{\text{reg1}}(g^i) = \text{reg1}[i]$$

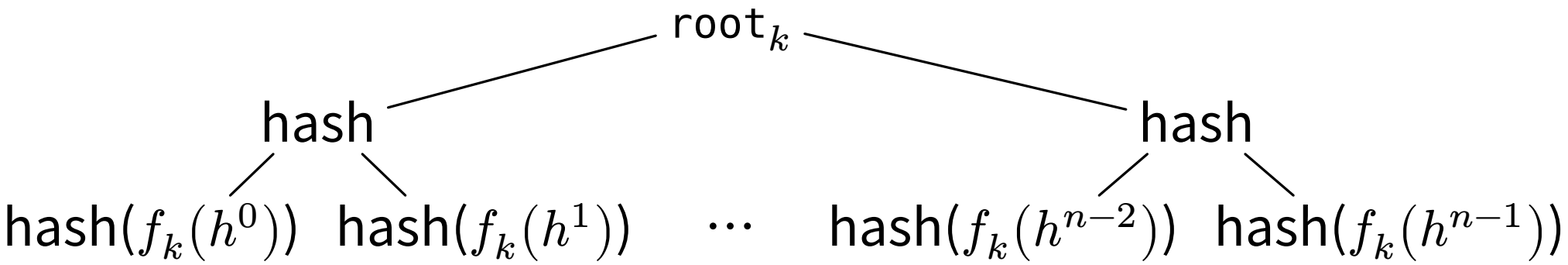
$$f_{\text{reg2}}(g^i) = \text{reg2}[i]$$

$$f_{\text{input}}(g^i) = \text{input}[i]$$

$$\vdots$$

14. Polynomial Commitment

Prover then evaluates $f_k(x)$ in a larger subgroup $\mathbb{L}$ with generator h and generate Merkle trees for those values. Then commit the root of the Merkle tree to the verifier so that verifier can check the values get from prover with Merkle proof in the following steps.



15. Constraints Polynomials

Now we need to add constraints to the columns of trace. For example, if the `add[i]` is `1`, then `reg1[i+1] = reg1[i] + reg2[i]`.

$$f_{\text{reg1}}(x \cdot g) = f_{\text{reg1}}(x) + f_{\text{reg2}}(x)$$

$x \cdot g$ is the next element in $\mathbb{G}$

$$\Rightarrow c_{\text{reg1\_add}}(x) = f_{\text{add}}(x) \cdot \left(f_{\text{reg1}}(x \cdot g) - \left(f_{\text{reg1}}(x) + f_{\text{reg2}}(x)\right)\right)$$

$$c_{\text{reg2\_add}}(x) = f_{\text{add}}(x) \cdot \left(f_{\text{reg2}}(x \cdot g) - f_{\text{reg2}}(x)\right)$$

reg2 should not change when add

$c_k(x) = 0 \forall x \in \mathbb{G}'$, where $\mathbb{G}'$ is the domain of the polynomial

the control columns also need to be constrained.

$$c_{\text{add}}(x) = f_{\text{add}}(x) \cdot (f_{\text{add}}(x) - 1)$$

add should be either 0 or 1

$$c_{\text{one\_hot}}(x) = f_{\text{set}}(x) + f_{\text{swap}}(x) + f_{\text{add}}(x) + f_{\text{mul}}(x) + f_{\text{nop}}(x)$$

exactly one of the operations should be 1

The initial state needs be be constrained as well.

$$c_{\text{reg1\_init}}(x) = f_{\text{reg1}}(x) - 0$$

$$c_{\text{reg2\_init}}(x) = f_{\text{reg2}}(x) - 0$$

Each constraint polynomial has its own domain, which is a subset of $\mathbb{G}$. In this VM, it can be categorized into three groups:

group	polynomials	description
all	$f_{\text{one\_hot}}(x)$, $f_{\text{add}}(x)$ etc.	domain is all elements in $\mathbb{G}$
first	$f_{\text{reg1\_init}}(x)$ and $f_{\text{reg2\_init}}(x)$	only the first element
trans	$f_{\text{r1\_add}}(x)$, $f_{\text{r1\_mul}}(x)$ etc.	except the last one element

Note: verifier can calculate the value of those constraint polynomials once the prover commits the roots of the trace polynomials.

16. Vanish Polynomials

Vanish polynomials are used to make sure that the prover constructs the constraint polynomials correctly.

$$u_k(x) = \prod_{g \in \mathbb{G}'} (x - g)$$

Note: verifier constructs the vanish polynomials by itself, so the prover cannot cheat by constructing a wrong vanish polynomial.

17. Composition Polynomial

$$p(x) = \frac{c(x)}{u(x)}$$

In $\mathbb{G}'$, $u(x) = 0$, but $c(x) = 0$, too. They can cancel each other out, so $p(x)$ still a good polynomial. However if the prover cheats on the constraint polynomials, for example $c(z) \neq 0, z \in \mathbb{G}'$, $c(x)$ can not be divided by $u(x)$ anymore and verifier can easily check it.

$$p(x) = \frac{c(x)}{u(x)}$$

In $\mathbb{G}'$, $u(x) = 0$, but $c(x) = 0$, too. They can cancel each other out, so $p(x)$ still a good polynomial. However if the prover cheats on the constraint polynomials, for example $c(z) \neq 0, z \in \mathbb{G}'$, $c(x)$ can not be divided by $u(x)$ anymore and verifier can easily check it.

$$p(x) = \sum_k \beta_k \frac{c_k(x)}{u_k(x)}$$

Note: prover needs to commit the composition polynomial as well

18. FRI

$$p_k(z) = g_k(z^2) + z \cdot h_k(z^2)$$
$$\Rightarrow p_{k+1}(z) = g_k(z) + \alpha_k \cdot h_k(z)$$

α_k is a random number generated by the verifier.

to check if $p_{k+1}(z)$ is correct

$$\begin{cases} p_k(z) &= g_k(z^2) + z \cdot h_k(z^2) \\ p_k(-z) &= g_k(z^2) - z \cdot h_k(z^2) \end{cases} \Rightarrow \begin{cases} g_{k(z^2)} \stackrel{?}{=} \frac{p_k(z)+p_k(-z)}{2} \\ h_{k(z^2)} \stackrel{?}{=} \frac{p_k(z)-p_k(-z)}{2z} \end{cases}$$

18.1 layer 0

$$
\begin{aligned}
p_0(z) &= az^4 + bz^3 + cz^2 + dz + e \\
&= \underbrace{(az^4 + cz^2 + e)}_{g_0(z') = az'^2 + cz' + e} + z \cdot \underbrace{(bz^2 + d)}_{h_0(z') = bz' + d}
\end{aligned}
$$

$$
\begin{aligned}
\Rightarrow p_1(z') &= g_0(z') + \alpha_0 \cdot h_0(z') \\
&= az'^2 + (c + \alpha_0 b)z' + (e + \alpha_0 d)
\end{aligned}
$$

18.2 layer 1

$$
\begin{aligned}
p_1(z') &= az'^2 + (c + \alpha_0 b)z' + (e + \alpha_0 d) \\
&= \underbrace{(az'^2 + e + \alpha_0 d)}_{g_1(z'') = az'' + e + \alpha_0 d} + z' \cdot \underbrace{(c + \alpha_0 b)}_{h_1(z'') = c + \alpha_0 b}
\end{aligned}
$$

$$
\begin{aligned}
\Rightarrow p_2(z'') &= g_1(z'') + \alpha_1 \cdot h_1(z'') \\
&= az'' + (e + \alpha_0 d + \alpha_1 c + \alpha_0 \alpha_1 b)
\end{aligned}
$$

18.3 layer 2

$$
\begin{aligned}
p_2(z'') &= az'' + (e + \alpha_0 d + \alpha_1 c + \alpha_0\alpha_1 b) \\
&= \underbrace{(e + \alpha_0 d + \alpha_1 c + \alpha_0\alpha_1 b)}_{g_2(z''')=e+\alpha_0 d+\alpha_1 c+\alpha_0\alpha_1 b} + z'' \cdot \underbrace{a}_{h_2(z''')=a}
\end{aligned}
$$

$$
\begin{aligned}
\Rightarrow p_3(z''') &= g_2(z''') + \alpha_2 \cdot h_2(z''') \\
&= e + \alpha_0 d + \alpha_1 c + \alpha_0\alpha_1 b + \alpha_2 a
\end{aligned}
$$

constant, no matter chooseing what z

18.4 References

- Berentsen, Aleksander and Lenzi, Jeremias and Nyffenegger, Remo, A Walk-through of a Simple Zk-STARK Proof (December 21, 2022).
- Wikipedia for theorems
- Gemini
- ChatGPT